

INFORME

PARTE 1 - CLASIFICACIÓN CON FASHION-MNIST

El objetivo fue entrenar una red neuronal para clasificar imágenes de ropa del dataset fashion_mnist y comparar el tiempo de entrenamiento entre CPU y GPU. La estructura utilizada fue una red neuronal densa con una capa de entrada a Flatten, una capa Dense de 128 neuronas con la función ReLU y la capa de salida con 10 neuronas y softmax.

Métrica	Valor CPU	Valor GPU
Épocas	5	5
Tiempo total de entrenamiento	39.7 segundos	17.2 segundos
Aceleración GPU (aproximada)	2.3x más rápida	

El entrenamiento en GPU fue más de dos veces más rápido que en CPU para este modelo relativamente simple. Esto demuestra que aun con redes densas y datasets medianos, el uso de GPU puede reducir de manera no menor el tiempo de entrenamiento.

PARTE 2 - CLASIFICACIÓN DE TEXTO CON MODELO PREENTRENADO (DISTILBERT)

En este segundo caso, el objetivo es clasificar comentarios tóxicos o no usando un modelo preentrenado con 10,000 ejemplos del dataset de comentarios, comparando el entrenamiento entre CPU y GPU. Se utilizó una muestra de 10,000 para poder tener una idea de cuánto demoraría, pero el total supera los 159,000 filas.

Métrica	Valor CPU	Valor GPU
Épocas	2	2
Tamaño del subconjunto	10,000 textos	10,000 textos
Tiempo total de entrenamiento	653.8 segundos	304.2 segundos
Aceleración GPU (aproximada)	2.15x más rápida	

Durante el entrenamiento del modelo, se observó que el uso de hilos de CPU se mantuvo constante en 28, lo que indica un uso estable del procesador a lo largo del proceso. En cuanto a la utilización del CPU en porcentaje, esta alcanza un máximo de 6.3%, lo que sugiere que el cuello de botella principal no estuvo en la capacidad de procesamiento, sino más bien en la carga y manejo del modelo.

La memoria utilizada, medida en megabytes, mostró variaciones significativas entre 5,000 - 10,000 MB, lo cual es consistente con la naturaleza dinámica del entrenamiento de modelo que manejan datos textuales o visuales. A pesar de estas variaciones, el sistema logró estabilizar el uso de memoria hacia el final del entrenamiento. En cuanto al uso del disco, esta se mantuvo constante sin picos ni comportamientos que indiquen una carga significativa sobre ese recurso.

CONCLUSIONES

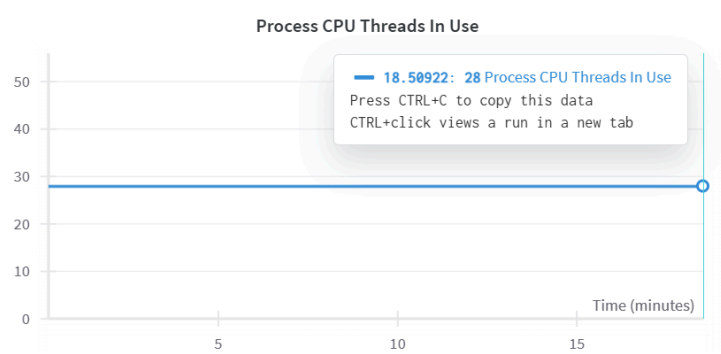
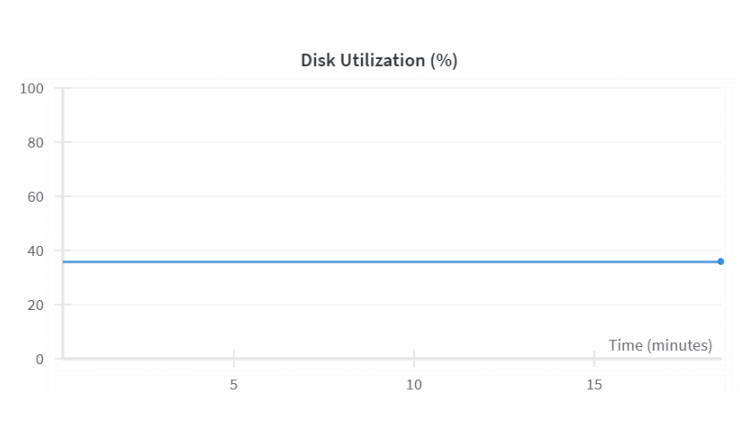
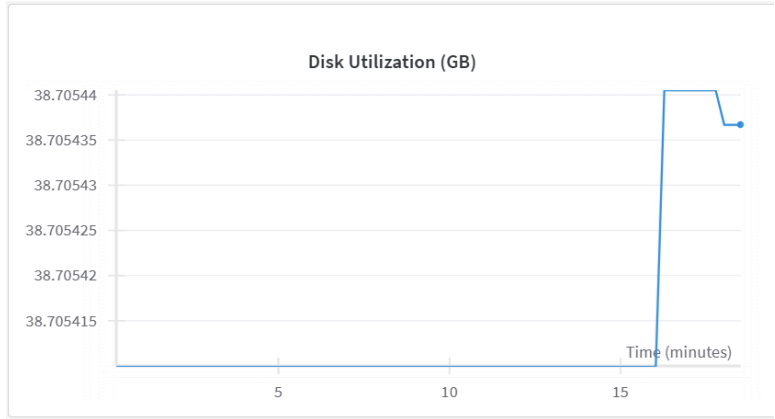
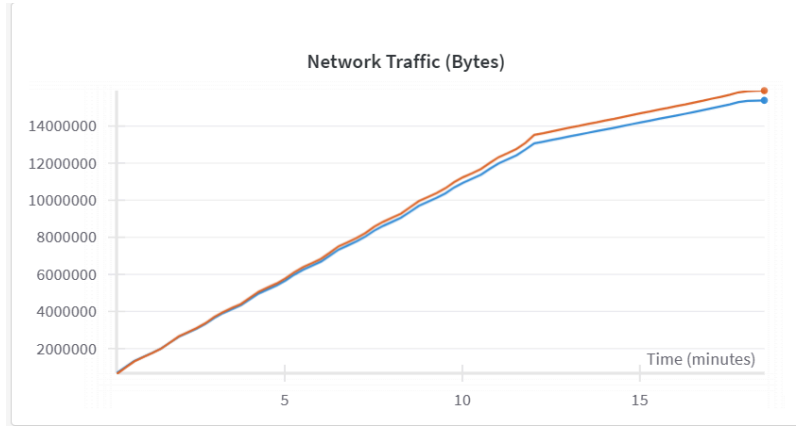
El uso de GPU permitió reducir el tiempo de entrenamiento tanto en el modelo simple de imágenes como en el modelo más complejo de texto. La aceleración fue aproximadamente 2.3x en ambos casos, lo que representa una ventaja clara en tareas de entrenamiento complejo.

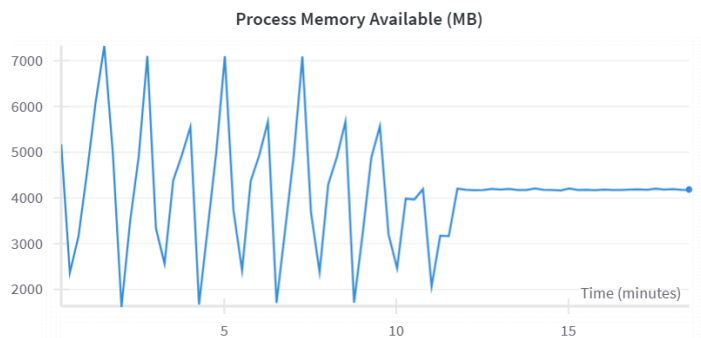
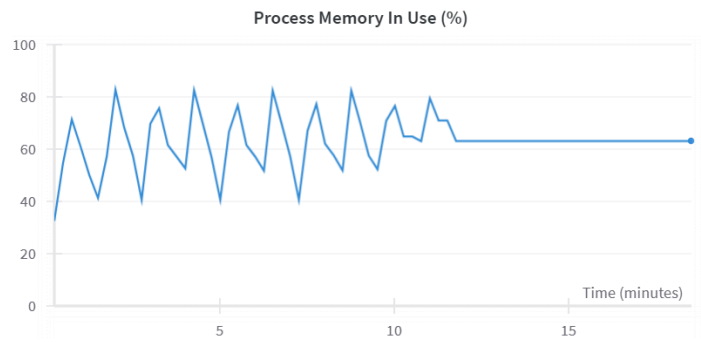
En definitiva la relevancia del uso de GPU quedó clara en la segunda parte de la tarea donde los tiempos en CPU superaban los 10 minutos. Usar GPU no solo permitió mayor eficiencia, sino también mayor viabilidad para escalar el modelo a conjuntos de datos más grandes. La diferencia de tiempo fue importante para lograr completar la tarea.

A pesar de lo anterior, uno de los principales retos fue la limitación computacional al trabajar con un dataset de más de 150,000 registros. Fue necesario seleccionar un subconjunto (10,000 muestras) para poder entrenar sin que el entorno colapse. Además, se requirió configurar correctamente el entorno en Google Colab y sincronizar Weights & Biases, lo que implicó autorizar un API.

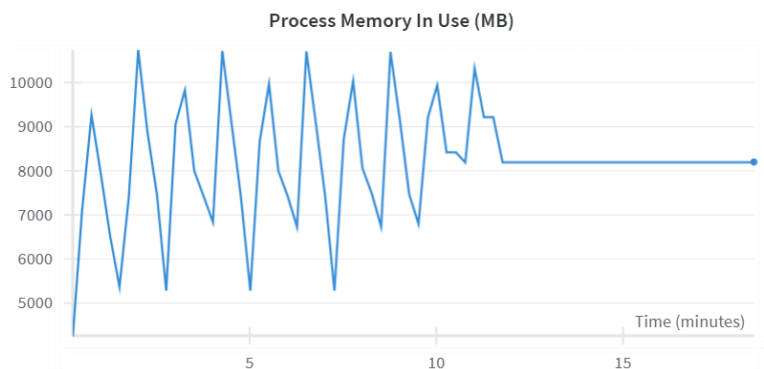
Finalmente, este segundo modelo tiene aplicaciones en múltiples sectores profesionales. En el ámbito financiero, por ejemplo, podría utilizarse para clasificar automáticamente comentarios de clientes, analizar noticias con lenguaje neutral, o identificar patrones en grandes bases de datos.

ANEXOS





⋮



Process CPU Utilization (%)

